Отчёт по программе «Хэллоуин» (кадровая графика, NURBS и Безье)

1. Общая идея решения

Программа **полностью вручную** рисует кадр «Хэллоуин» на холсте `1920×1080`:

- создаётся тёмный фон;
- при помощи **NURBS** рисуется круглая луна, из которой затем вырезается **серп**, на видимой части добавляются 3 кратера;
- снизу растут **5 ветвей деревьев** (кривые Безье) с боковыми тонкими ответвлениями;
- по центру внизу находится **тыковка (тыква) как NURBS-овал**, поверх которой рисуются:
  - 5 «долек»-рёбер (кривые Безье 2-го порядка),
  - глаза-треугольники,
  - треугольный нос (тёмный, как требует задание),
  - открытый рот (2 кривые Безье + многоугольная заливка) и треугольные зубы;
- на кочерыжке сидит **ворон**:
  - тело — замкнутая кривая Безье;
  - голова — закрашенный круг;
  - крыло — отдельный контур Безье с заливкой;
  - глаз и зрачок — круги;
  - клюв — две кривые Безье;
  - лапы и пальцы — кривые Безье;
  - несколько перьев на крыле — тоже квадратичные кривые Безье.

Отрисовка делается поверх `numpy`-массива, а все базовые операции — своими функциями: scanline-заливка, Брезенхем, собственные NURBS и Безье.

---

2. Импорты и глобальные параметры

```
import numpy as np
import matplotlib.pyplot as plt
import random
```

- `numpy` — удобная работа с массивами и числом, хранение холста.

- `matplotlib` — только для финального `imshow` и `savefig`.
- `random` — рандомные кратеры на луне и разветвления веток.

Затем задаются размеры холста и основные цвета:

```
canvas_width, canvas_height = 1920, 1080

canvas_color = (30/255, 33/255, 36/255)        # почти черный
moon_color   = (255/255, 253/255, 209/255)     # бледно-желтый
crater_color = (180/255, 180/255, 180/255)     # светло-серый
pumpkin_color = (255/255, 140/255, 0/255)      # оранжевый
...
```

Все цвета заданы в формате `[0,1]`, чтобы их напрямую использовать в `imshow`.

---

3. Математические функции: B-сплайн, NURBS, Безье

3.1. `basis_function` — базисная функция B-сплайна

```
def basis_function(i, p, u, T):
    if p == 0:
        return 1.0 if T[i] <= u < T[i+1] else (1.0 if u == T[-1] and i ==
len(T) - 2 else 0.0)
    else:
        left = 0.0
        right = 0.0

        if T[i+p] - T[i] != 0:
            left = (u - T[i]) / (T[i+p] - T[i]) * basis_function(i, p-1, u,
T)

        if T[i+p+1] - T[i+1] != 0:
            right = (T[i+p+1] - u) / (T[i+p+1] - T[i+1]) *
basis_function(i+1, p-1, u, T)

        return left + right
```

Это рекурсивная реализация базисной функции B-сплайна (N_{i,p}(u)) по формуле де Бура:

- при `p == 0` — кусочная константная функция на отрезке `[T[i], T[i+1])`;

- при `p > 0` — линейная комбинация двух базисов степени `p-1` с защитой от деления на ноль для совпадающих узлов.

3.2. `nurbs_curve` — вычисление NURBS-кривой

```
def nurbs_curve(control_points, weights, knot_vector, degree,
num_samples=100):
    curve_points = []

    for u in np.linspace(knot_vector[0], knot_vector[-1], num_samples):
        numerator = np.array([0.0, 0.0])
        denominator = 0.0

        for i in range(len(control_points)):
            N = basis_function(i, degree, u, knot_vector)
            numerator += weights[i] * N * np.array(control_points[i])
            denominator += weights[i] * N

        if denominator > 1e-10:
            curve_points.append(numerator / denominator)

    return np.array(curve_points)
```

Функция считает точки NURBS-кривой:

[
C(u) = \frac{\sum*i N*{i,p}(u) w*i P_i}{\sum_i N*{i,p}(u) w_i}
]

- `control_points` — контрольные точки (P_i),
- `weights` — веса (w_i),
- `knot_vector` и `degree` задают базис (N_{i,p}),
- `u` пробегает диапазон узлов.

Эта функция применяется для:

- круга-луны;
- контура тыквы;
- кочерыжки.

3.3. `bezier_curve` — кривая Безье произвольного порядка

```
def bezier_curve(control_points, num_samples=100):
    curve_points = []
    degree = len(control_points) - 1

    for t in np.linspace(0, 1, num_samples):
        point = np.array([0.0, 0.0])

        for i, cp in enumerate(control_points):
            C = 1
            for k in range(i):
                C = C * (degree - k) // (k + 1)

            B = C * (t ** i) * ((1 - t) ** (degree - i))
            point += B * np.array(cp)

        curve_points.append(point)

    return np.array(curve_points)
```

Здесь:

- `degree = n` — порядок кривой;
- для кажой контрольной точки считается биномиальный коэффициент (C(n, i));
- итоговая точка — сумма (C(n, i) t^i (1-t)^{n-i} P_i).

Эта функция используется **повсюду**:

- для ветвей деревьев;
- многоугольного тела ворона;
- крыла, лап, пальцев, клюва;
- рта;
- долек-ребер на тыкве.

---

4. Графические примитивы: заливка и линии

4.1. `fill_polygon_scanline` — заливка многоугольника

```
def fill_polygon_scanline(canvas, points, color):
    if len(points) < 3:
        return
```

```
    points = np.array(points, dtype=int)
    min_y = max(0, np.min(points[:, 1]))
    max_y = min(canvas.shape[0] - 1, np.max(points[:, 1]))

    for y in range(min_y, max_y + 1):
        intersections = []

        for i in range(len(points)):
            p1 = points[i]
            p2 = points[(i + 1) % len(points)]
            ...
            if (y1 <= y < y2) or (y2 <= y < y1):
                ...
                x = x1 + (y - y1) * (x2 - x1) / (y2 - y1)
                intersections.append(int(x))

        if len(intersections) >= 2:
            intersections.sort()
            for j in range(0, len(intersections) - 1, 2):
                x_start = max(0, intersections[j])
                x_end = min(canvas.shape[1] - 1, intersections[j + 1])
                canvas[y, x_start:x_end + 1] = color
```

Классический алгоритм «сканирующей строки»:

- по каждой строке `y` ищем пересечения контура с горизонталью;
- сортируем `x`-координаты пересечений;
- заполняем отрезки `[x0, x1]`, `[x2, x3]` и т.д.

Применяется для заливки:

- луны;
- тыквы;
- внутренней полости рта;
- глаз, носа;
- тела ворона, крыла;
- других фигур.

4.2. `fill_circle_scanline` — заливка круга

```
def fill_circle_scanline(canvas, center_x, center_y, radius, color):
    center_x, center_y = int(center_x), int(center_y)
    radius = int(radius)
```

```
    for y in range(max(0, center_y - radius), min(canvas.shape[0], center_y
+ radius + 1)):
        dy = y - center_y
        if abs(dy) <= radius:
            dx = int(np.sqrt(radius**2 - dy**2))
            x_start = max(0, center_x - dx)
            x_end = min(canvas.shape[1] - 1, center_x + dx)
            canvas[y, x_start:x_end + 1] = color
```

Для каждой строки внутри диапазона по `y` вычисляется круг:

$$
\begin{cases}
(x - x_c)^2 + (y - y_c)^2 \le R^2 \\
\Rightarrow \\
x \in \left[x_c - \sqrt{R^2 - dy^2},\, x_c + \sqrt{R^2 - dy^2}\right]
\end{cases}
$$

Используется для:

- выреза серпа (кругом цвета фона);
- кратеров;
- концов кочерыжки;
- головы ворона, глаза, зрачка.

4.3. `draw_line_bresenham` и `draw_bezier_with_line`

```
def draw_line_bresenham(canvas, x0, y0, x1, y1, color, linewidth=1):
    ...
    # основной Брезенхем
    while True:
        points.append((x, y))
        ...
    # толщина линии
    for px, py in points:
        for dx in range(-linewidth//2, linewidth//2 + 1):
            for dy in range(-linewidth//2, linewidth//2 + 1):
                nx, ny = px + dx, py + dy
                ...
                canvas[ny, nx] = color
```

`draw_bezier_with_line` просто пробегает по точкам кривой и соединяет соседние точки Брезенхемом:

```
def draw_bezier_with_line(canvas, points, color, linewidth=1):
    for i in range(len(points) - 1):
        draw_line_bresenham(canvas, points[i][0], points[i][1],
                            points[i+1][0], points[i+1][1], color, linewidth)
```

Это даёт «толстые» плавные линии по результатам кривых Безье и NURBS.

---

5. Создание холста

```
canvas = np.zeros((canvas_height, canvas_width, 3))
for c in range(3):
    canvas[:, :, c] = canvas_color[c]
```

Здесь создаётся чёрно-серый фон на всём изображении.

---

6. Луна как NURBS-круг + кратеры

6.1. Положение и радиус

```
moon_radius   = int(min(canvas_width, canvas_height) * 0.35)
moon_center_x = canvas_width // 2
moon_center_y = canvas_height // 2 - 250
```

Крупная луна почти по центру, но смещена вверх.

6.2. NURBS-круг (полная луна)

```
control_points_moon = np.array([
    [moon_center_x + moon_radius, moon_center_y],
    [moon_center_x + moon_radius, moon_center_y + moon_radius],
    [moon_center_x,               moon_center_y + moon_radius],
    [moon_center_x - moon_radius, moon_center_y + moon_radius],
    [moon_center_x - moon_radius, moon_center_y],
    [moon_center_x - moon_radius, moon_center_y - moon_radius],
    [moon_center_x,               moon_center_y - moon_radius],
    [moon_center_x + moon_radius, moon_center_y - moon_radius],
    [moon_center_x + moon_radius, moon_center_y]
], dtype=float)
```

```
moon_weights = np.array([1.0, np.sqrt(2)/2, 1.0, np.sqrt(2)/2,
                         1.0, np.sqrt(2)/2, 1.0, np.sqrt(2)/2, 1.0])
moon_knot = np.array([0, 0, 0,
                      0.25, 0.25,
                      0.5, 0.5,
                      0.75, 0.75,
                      1, 1, 1], dtype=float)
moon_degree = 2
```

Это классическая аппроксимация окружности NURBS:

- 8 «опорных» точек + дубль первой;
- диагональные точки имеют вес (\sqrt{2}/2);
- knot-vector с повторениями для замыкания.

Дальше кривая вычисляется и заполняется:

```
moon_points = nurbs_curve(...)
if len(moon_points) > 0:
    fill_polygon_scanline(canvas, moon_points, moon_color)
    draw_bezier_with_line(canvas, moon_points, moon_color, linewidth=2)
```

6.3. Кратеры СРАЗУ на луне

После последней версии серпа в этом коде вырез луны закомментирован, поэтому кратеры рисуются на полной круговой луне:

```
crater_color = (180/255, 180/255, 180/255)

craters_drawn = 0
max_attempts = 200

crescent_cut_radius = int(moon_radius * 0.95)
crescent_cut_center_x = moon_center_x + int(moon_radius * 0.35)
crescent_cut_center_y = moon_center_y

attempts = 0
while craters_drawn < 3 and attempts < max_attempts:
    attempts += 1

    angle = random.uniform(np.pi/2, 3*np.pi/2)
    crater_dist   = random.uniform(moon_radius * 0.3, moon_radius * 0.8)
    crater_radius = random.uniform(moon_radius * 0.1, moon_radius * 0.2)
```

```
    x = moon_center_x + crater_dist * np.cos(angle)
    y = moon_center_y + crater_dist * np.sin(angle)

    # проверка внутри луны, вне выреза, левее центра
    ...
    fill_circle_scanline(canvas, x, y, crater_radius, crater_color)
    craters_drawn += 1
```

Логика:

- берём угол из левой половины луны (`[π/2, 3π/2]`);
- проверяем, что точка внутри большого круга;
- теоретически код готов к варианту с серпом (проверка выреза и закомментированный `fill_circle_scanline` для вырезания), но в текущем варианте серп не применяется.

---

7. Ветви деревьев

7.1. Основные параметры

```
branch_base_y = canvas_height - 50
mid_moon_y = moon_center_y

num_branches = 5
branch_width = 8
```

5 толстых стволов, идущих от земли до уровня луны.

7.2. Основные ветви (квадратичные Безье)

```
for branch_idx in range(num_branches):
    start_x = canvas_width // 5 + (branch_idx * canvas_width // 5)
    start_x = max(100, min(canvas_width - 100, start_x))
    start_y = branch_base_y

    end_x = start_x + random.randint(-50, 50)
    end_y = mid_moon_y

    num_control_points = 3
    control_points_branch = [[start_x, start_y]]

    mid_x = start_x + random.randint(-30, 30)
    mid_y = (start_y + end_y) // 2
```

```
    control_points_branch.append([mid_x, mid_y])
    control_points_branch.append([end_x, end_y])

    branch_points = bezier_curve(control_points_branch, num_samples=300)
    draw_bezier_with_line(canvas, branch_points, branch_color, branch_width)
```

Каждая ветка:

- определена 3 точками → Безье 2-го порядка;
- слегка шумится в середине по X для естественности.

7.3. Боковые ветвления

```
for sub_branch in range(2):
    start_idx = random.randint(int(len(branch_points) * 0.3),
int(len(branch_points) * 0.7))
    start_point = branch_points[start_idx]
    angle = random.choice([-1, 1])
    sub_length = random.randint(80, 150)
    sub_control_points = [
        start_point,
        [start_point[0] + angle * sub_length * 0.5, start_point[1] -
sub_length * 0.3],
        [start_point[0] + angle * sub_length,       start_point[1] -
sub_length * 0.7]
    ]
    ...
```

У каждой большой ветви по 2 боковые тонкие ветки, уходящие в стороны и вверх.

---

8. Тыква: NURBS-контур и рёбра («дольки»)

8.1. Контур тыквы (NURBS)

```
pumpkin_center_x = canvas_width // 2
pumpkin_center_y = canvas_height // 2 + 400
pumpkin_radius   = 180

pumpkin_cp = np.array([
    [pumpkin_center_x + pumpkin_radius * 1.1, pumpkin_center_y],
    [pumpkin_center_x + pumpkin_radius * 1.1, pumpkin_center_y +
pumpkin_radius * 0.8],
```

```
    [pumpkin_center_x,                          pumpkin_center_y +
pumpkin_radius * 1.2],
    [pumpkin_center_x - pumpkin_radius * 1.1, pumpkin_center_y +
pumpkin_radius * 0.8],
    [pumpkin_center_x - pumpkin_radius * 1.1, pumpkin_center_y],
    [pumpkin_center_x - pumpkin_radius * 1.1, pumpkin_center_y -
pumpkin_radius * 0.8],
    [pumpkin_center_x,                          pumpkin_center_y -
pumpkin_radius * 1.2],
    [pumpkin_center_x + pumpkin_radius * 1.1, pumpkin_center_y -
pumpkin_radius * 0.8],
    [pumpkin_center_x + pumpkin_radius * 1.1, pumpkin_center_y]
], dtype=float)

pumpkin_weights = np.array([1.0, 2.0, 1.0, 2.0, 1.0, 2.0, 1.0, 2.0, 1.0])
pumpkin_knot    = np.array([0,0,0, 0.25,0.25, 0.5,0.5, 0.75,0.75, 1,1,1],
dtype=float)
pumpkin_degree  = 2

pumpkin_points = nurbs_curve(...)
fill_polygon_scanline(canvas, pumpkin_points, pumpkin_color)
draw_bezier_with_line(canvas, pumpkin_points, (200/255, 100/255, 0/255),
linewidth=3)
```

Форма — слегка вытянутая и «приплюснутая» тыква.

8.2. Дольки-ребра тыквы (5 кривых Безье)

Специальная функция:

```
def draw_pumpkin_ridge(canvas, p0, p1, p2, color, linewidth=8):
    curve = bezier_curve([p0, p1, p2], num_samples=120)
    draw_bezier_with_line(canvas, curve, color, linewidth)

ridge_color = (200/255, 100/255, 0/255)  # тёмно-оранжевый
```

Каждая «долька» — это просто квадратичная Безье (3 точки):

```
# левая внешняя
draw_pumpkin_ridge(
    canvas,
    p0 = [pumpkin_center_x - pumpkin_radius * 0.82,  pumpkin_center_y -
pumpkin_radius * 0.8],
    p1 = [pumpkin_center_x - pumpkin_radius * 1.05,  pumpkin_center_y -
pumpkin_radius * 0.05],
```

```
    p2 = [pumpkin_center_x - pumpkin_radius * 0.78,  pumpkin_center_y +
pumpkin_radius * 0.8],
    color = ridge_color,
    linewidth = 14
)

# левая внутренняя
draw_pumpkin_ridge(
    canvas,
    p0 = [pumpkin_center_x - pumpkin_radius * 0.45,  pumpkin_center_y -
pumpkin_radius * 0.97],
    p1 = [pumpkin_center_x - pumpkin_radius * 0.80,  pumpkin_center_y -
pumpkin_radius * 0.10],
    p2 = [pumpkin_center_x - pumpkin_radius * 0.40,  pumpkin_center_y +
pumpkin_radius * 0.8],
    color = ridge_color,
    linewidth = 10
)

# правая внутренняя
draw_pumpkin_ridge(
    canvas,
    p0 = [pumpkin_center_x + pumpkin_radius * 0.45,  pumpkin_center_y -
pumpkin_radius * 0.97],
    p1 = [pumpkin_center_x + pumpkin_radius * 0.8,   pumpkin_center_y -
pumpkin_radius * 0.10],
    p2 = [pumpkin_center_x + pumpkin_radius * 0.40,  pumpkin_center_y +
pumpkin_radius * 0.8],
    color = ridge_color,
    linewidth = 10
)

# правая внешняя
draw_pumpkin_ridge(
    canvas,
    p0 = [pumpkin_center_x + pumpkin_radius * 0.82,  pumpkin_center_y -
pumpkin_radius * 0.8],
    p1 = [pumpkin_center_x + pumpkin_radius * 1.05,  pumpkin_center_y -
pumpkin_radius * 0.05],
    p2 = [pumpkin_center_x + pumpkin_radius * 0.78,  pumpkin_center_y +
pumpkin_radius * 0.8],
    color = ridge_color,
    linewidth = 14
)
```

И центральная прямая «долька» по центру тыквы:

```
draw_pumpkin_ridge(
    canvas,
    p0=[pumpkin_center_x, pumpkin_center_y - pumpkin_radius],
    p1=[pumpkin_center_x, pumpkin_center_y - pumpkin_radius * 0.18],
    p2=[pumpkin_center_x, pumpkin_center_y + pumpkin_radius],
    color=ridge_color,
    linewidth=14
)
```

Итого — 5 рёбер: `(( | ))` по форме.

---

9. Кочерыжка тыквы (NURBS-кривая)

```
stem_cp = np.array([
    [pumpkin_center_x,          pumpkin_center_y - pumpkin_radius * 1.1],
    [pumpkin_center_x + 30,     pumpkin_center_y - pumpkin_radius * 1.15],
    [pumpkin_center_x + 60,     pumpkin_center_y - pumpkin_radius * 1.1],
    [pumpkin_center_x + 90,     pumpkin_center_y - pumpkin_radius * 1.0],
    [pumpkin_center_x + 110,    pumpkin_center_y - pumpkin_radius * 0.9]
], dtype=float)
...
stem_points = nurbs_curve(..., degree=3, ...)
draw_bezier_with_line(canvas, stem_points, stem_color, linewidth=16)
```

Плавная «загнутая» ножка, затем конец закругляется кругом:

```
stem_top_x = stem_points[-1][0]
stem_top_y = stem_points[-1][1]
fill_circle_scanline(canvas, stem_top_x, stem_top_y, stem_top_radius,
stem_color)
```

---

10. Лицо тыквы: глаза, нос, рот и зубы

10.1. Глаза

```
eye_left_x = pumpkin_center_x - 60
eye_left_y = pumpkin_center_y - 40
...
```

```
left_eye_points = np.array([
    [eye_left_x - 25, eye_left_y + 20],
    [eye_left_x + 25, eye_left_y + 20],
    [eye_left_x,      eye_left_y - 25]
])
fill_polygon_scanline(canvas, left_eye_points, (0, 0, 0))
```

Два треугольника-«классические» глаза тыквы.

10.2. Нос (треугольник)

```
nose_x = pumpkin_center_x
nose_y = pumpkin_center_y
nose_size = 30

nose_points = np.array([
    [nose_x - nose_size, nose_y - nose_size],
    [nose_x + nose_size, nose_y - nose_size],
    [nose_x,             nose_y + nose_size]
])
fill_polygon_scanline(canvas, nose_points, (0, 0, 0))
```

Нос явно НЕ круглый, как требует условие.

10.3. Открытый рот с зубами

Границы рта — две квадратичные кривые Безье:

```
mouth_top_cp = [
    [pumpkin_center_x - 90, pumpkin_center_y + 40],
    [pumpkin_center_x,      pumpkin_center_y + 10],
    [pumpkin_center_x + 90, pumpkin_center_y + 40]
]

mouth_bottom_cp = [
    [pumpkin_center_x - 90, pumpkin_center_y + 40],
    [pumpkin_center_x,      pumpkin_center_y + 120],
    [pumpkin_center_x + 90, pumpkin_center_y + 40]
]

mouth_top_curve    = bezier_curve(mouth_top_cp, num_samples=140)
mouth_bottom_curve = bezier_curve(mouth_bottom_cp, num_samples=140)
mouth_polygon      = np.vstack([mouth_top_curve, mouth_bottom_curve[::-1]])
fill_polygon_scanline(canvas, mouth_polygon, mouth_inside_color)
```

Дальше зубы:

```
tooth_color = (245/255, 245/255, 220/255)
upper_y = pumpkin_center_y + 40
lower_y = pumpkin_center_y + 90
tooth_w = 26
tooth_h = 28

# верхние зубы
for cx in [center_x - 60, -20, +20, +60]:
    tooth_pts = [
        [cx - tooth_w/2, upper_y],
        [cx + tooth_w/2, upper_y],
        [cx,             upper_y + tooth_h]
    ]
    fill_polygon_scanline(...)

# нижние зубы
for cx in [center_x - 40, center_x, center_x + 40]:
    ...
    [cx, lower_y - tooth_h]
```

Получается открытый рот с несколькими треугольными зубами сверху и снизу.

---

11. Ворон на кочерыжке

11.1. Положение и размеры

```
raven_x = stem_top_x - 50
raven_y = stem_top_y - stem_top_radius - 200

raven_body_width  = 200
raven_body_height = 160
raven_head_radius = 70
```

Ворон сидит немного левее от конца кочерыжки.

11.2. Тело ворона (замкнутая кривая Безье)

```
body_control_points = [
    [raven_x,                    raven_y - H * 0.5],
    [raven_x + W * 0.6,          raven_y - H * 0.4],
```

```
    [raven_x + W * 0.8,              raven_y],
    [raven_x + W * 0.6,              raven_y + H * 0.4],
    [raven_x,                        raven_y + H * 0.5],
    [raven_x - W * 0.6,              raven_y + H * 0.4],
    [raven_x - W * 0.8,              raven_y],
    [raven_x - W * 0.6,              raven_y - H * 0.4],
    [raven_x,                        raven_y - H * 0.5]
]
body_points = bezier_curve(body_control_points, num_samples=300)
fill_polygon_scanline(canvas, body_points, raven_color)
```

Даёт «каплеобразное» тело.

11.3. Голова

```
head_center_x = raven_x + raven_body_width * 0.5
head_center_y = raven_y - raven_body_height * 0.4
fill_circle_scanline(canvas, head_center_x, head_center_y,
raven_head_radius, raven_color)
```

Большой круг над телом.

11.4. Крыло

```
wing_control_points = [
    [raven_x - W * 0.3, raven_y - H * 0.2],
    [raven_x - W * 0.9, raven_y - H * 0.6],
    [raven_x - W * 1.3, raven_y - H * 0.3],
    [raven_x - W * 1.4, raven_y + H * 0.25],
    [raven_x - W * 0.7, raven_y + H * 0.4]
]
wing_points = bezier_curve(wing_control_points, num_samples=250)
fill_polygon_scanline(canvas, wing_points, wing_color)
draw_bezier_with_line(canvas, wing_points, (150/255,150/255,150/255),
linewidth=3)
```

Крыло обрисовано отдельно и светло подсвечено (выделено), как требует условие.

11.5. Глаз и зрачок

```
eye_x = head_center_x + raven_head_radius * 0.4
eye_y = head_center_y - raven_head_radius * 0.2
eye_radius = 20
fill_circle_scanline(canvas, eye_x, eye_y, eye_radius, eye_color)
```

```
pupil_radius = 8
fill_circle_scanline(canvas, eye_x + eye_radius * 0.5, eye_y, pupil_radius,
(0, 0, 0))
```

Жёлтый круг + чёрный кружок делает глаз живым.

11.6. Клюв

```
beak_control_points = [
    [head_center_x + r * 0.95, head_center_y - r * 0.1],
    [head_center_x + r * 1.4,  head_center_y - r * 0.05],
    [head_center_x + r * 1.6,  head_center_y]
]
beak_points = bezier_curve(...)
draw_bezier_with_line(canvas, beak_points, beak_color, linewidth=8)

beak_bottom_control_points = [...]
...
```

Две кривые Безье — верхняя и нижняя часть клюва.

11.7. Лапы + пальцы

```
left_leg_control_points = [
    [raven_x - W * 0.3, raven_y + H * 0.2],
    [raven_x - W * 0.4, raven_y + H * 0.7],
    [raven_x - W * 0.25,raven_y + H * 0.85]
]
left_leg_points = bezier_curve(...)
draw_bezier_with_line(canvas, left_leg_points, beak_color, linewidth=7)
```

Пальцы — короткие кривые Безье, отходящие от последней точки:

```
for toe_angle in [-0.3, 0, 0.3]:
    toe_control_points = [
        left_leg_points[-1],
        [last_x + toe_angle * 35, last_y + 25],
        [last_x + toe_angle * 50, last_y + 40]
    ]
    ...
```

Правая лапа сделана аналогично.

11.8. Перья на крыле

```
feather_color = (40/255, 40/255, 40/255)

feather_cps = [
    [...],  # перо ближе к хвосту
    [...],  # среднее
    [...]   # ближе к телу
]

for cps in feather_cps:
    f_curve = bezier_curve(cps, num_samples=100)
    draw_bezier_with_line(canvas, f_curve, feather_color, linewidth=5)
```

Каждое перо — квадратичная кривая Безье (3 точки), проходящая по крылу. Таким образом, у ворона есть минимум 3 перышка, как требует бонус.

---

12. Финальный вывод кадра

В конце код просто показывает результат:

```
fig, ax = plt.subplots(figsize=(canvas_width/100, canvas_height/100),
dpi=100)
ax.imshow(canvas)
ax.axis('off')
plt.tight_layout(pad=0)
plt.savefig('halloween_pumpkin_moon.png', dpi=100, bbox_inches='tight')
plt.show()

print("Хэллоуин рисунок готов!")
print(f"Размер холста: {canvas.shape}")
```

- `canvas` — итоговый кадр с луной, ветвями, тыквой и вороном;
- картинка сохраняется в файл `halloween_pumpkin_moon.png`;
- печатаются служебные сообщения.

---

13. Соответствие требованиям задания

В итоге программа реализует:

- **фон** — тёмный;
- **луна** — NURBS-круг, вручную закрашенный, с кратерами;
- **ветви** — минимум 5, кривые Безье порядка ≥ 2, с ответвлениями;
- **тыква**:
  - сама — NURBS-фигура;
  - нос — треугольник, тёмный;
  - «дольки» — минимум 4, всего 5 рёбер, кривые Безье второго порядка;
  - рот — открытый, с зубами (многоугольная область + треугольные зубы);
- **ворон**:
  - тело, крыло, голова — построены кривыми Безье + круги;
  - клюв — кривые Безье;
  - глаз — круглый, жёлтый;
  - две лапки с пальцами;
  - минимум 3 пера — отдельные кривые Безье;
  - цвет — тёмный, но отличим от фона;
  - сидит на кочерыжке тыквы.

Всё рисование реализовано без готовых высокоуровневых графических библиотек: через собственные функции NURBS, Безье, scanline и Брезенхема.